TCSS 321 – Discrete Structures

Bubble Sort – Proof of Correctness

Sorting

Sort a List

Input: A sequence of integers $A = (a_1, a_2, \ldots, a_n)$.

Output: A' a permutation of A such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

Definition: A permutation of a sequence (or set) are the elements of the sequence in a (potentially) different order.

- The definition of permutation implies two important properties:
  1. All the elements of the original sequence are included in the permutation (no missing elements).
  2. No additional elements or duplicate elements have been added (no extra elements).

- Sorting a sequence of integers is a central problem in computer science.
  - There are several well-known sorting algorithms.

Bubble Sort

Sort a List
Input: A sequence of integers $A = (a_1, a_2, \ldots, a_n)$.
Output: A' a permutation of A such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

- The simplest sorting algorithm is called bubble sort.

```
BubbleSort (A = (a_1, a_2, ..., a_n))
        For i = 1 to n − 1:
                For j = 1 to n − i:
                        If a_j > a_{j+1} then Swap(j, j + 1).
                Endfor
        Endfor
Return.
```

Bubble Sort

```
BubbleSort (A = (a_1, a_2, ..., a_n))
        For i = 1 to n − 1:
                For j = 1 to n − i:
                        If a_j > a_{j+1} then Swap(j, j + 1).
                Endfor
        Endfor
Return.
```

- There are two loops in this algorithm. To prove it correct we will use two loop invariants.
  - We prove the loop invariant for the inner loop first.
  - We will use the loop invariant for the inner loop to prove the invariant for the outer loop.

- Can we identify the two loop invariants?

Inner Loop

Theorem: After iteration j of the inner loop the maximum element of $(a_1, a_2, \dots, a_{j+1})$ is in position $j + 1$.

Proof: (by induction on j)

Base case: $(j = 0)$

Iteration $j = 0$ is prior to the loop. In this step the maximum element of (a_1) is in position 1.

Inductive Hypothesis: Assume prior to iteration $j + 1$ that the maximum element of $(a_1, a_2, \dots, a_{j+1})$ is in position $j + 1$.

Inductive Step: $(j \geq 0)$

We need to show that after iteration $j + 1$ that the maximum element of $(a_1, a_2, \dots, a_{j+2})$ is in position $j + 2$.

We know by assumption that the maximum element of $(a_1, a_2, \dots, a_{j+1})$ is in position $j + 1$. Either this is the maximum element of $(a_1, a_2, \dots, a_{j+2})$ or element a_{j+2} is. In this iteration we compare a_{j+1} to a_{j+2} and swap them if $a_{j+1} > a_{j+2}$. So after this iteration the maximum element of $(a_1, a_2, \dots, a_{j+2})$ is in position $j + 2$.

So by induction after iteration j of the loop the maximum element of $(a_1, a_2, \dots, a_{j+1})$ is in position $j + 1$. ■

Outer Loop

Theorem: After iteration i of the outer loop the i maximum elements of $(a_1, a_2, \dots, a_n)$ are in ascending order in positions $n - i + 1$ to n.

Proof: (by induction on i)

Base case: ($i = 0$)

Prior to the loop the 0 maximum elements of $(a_1, a_2, \dots, a_n)$ are in ascending order in no positions (positions $n + 1$ to n).

Inductive Hypothesis: Assume prior to iteration $i + 1$ that the i maximum elements of $(a_1, a_2, \dots, a_n)$ are in ascending order in positions $n - i + 1$ to n.

Inductive Step: ($i \geq 0$)

By assumption we know that the i maximum elements of $(a_1, a_2, \dots, a_n)$ are in ascending order in positions $n - i + 1$ to n. We need to show that the maximum element of $(a_1, a_2, \dots, n - i)$ is in position $n - i$ after iteration $i + 1$.

By the inner loop invariant we know after iteration j of the inner loop that the maximum element of $(a_1, a_2, \dots, a_{j+1})$ is in position $j + 1$. In particular, iteration $i + 1$ the outer loop ends after iteration $n - i - 1$ of the inner loop. So after iteration $n - i - 1$ of the inner loop the maximum element of $(a_1, a_2, \dots, a_{n-i-1+1})$ is in position $n - i - 1 + 1$.

By induction we have shown that after iteration i of the outer loop the i maximum elements of $(a_1, a_2, \dots, a_n)$ are in ascending order in positions $n - i + 1$ to n. ■

Proof of Correctness

Theorem: The Bubble Sort algorithm is correct for the Sort problem.

Proof: Assume an arbitrary input sequence $A = (a_1, a_2, \ldots, a_n)$.

By the outer loop invariant we know that after iteration i of the outer loop the i maximum elements of $(a_1, a_2, \ldots, a_n)$ are in ascending order in positions $n - i + 1$ to n.

In particular, after iteration $n - 1$ of the outer loop the $n - 1$ maximum elements of $(a_1, a_2, \ldots, a_n)$ are in ascending order in positions 2 to n. The remaining element is the minimum element and it is in position 1.

We return after iteration $n - 1$ so we return a permutation of A in ascending order. ■

TCSS 321 – Discrete Structures

Bubble Sort – Run Time Analysis

Number of Operations

```
BubbleSort (A = (a_1, a_2, ..., a_n))
        For i = 1 to n − 1:
                For j = 1 to n − i:
                        If a_j > a_{j+1} then Swap(j, j + 1).
                Endfor
        Endfor
Return.
```

1 method call
1 ass/inc, 1 sub and 1 comp
1 ass/inc, 1 sub and 1 comp
1 comp, 2 adds and 1 swap
3 operations
3 operations
1 return

- How many operations are carried out on each line of this algorithm?
- Can we express the cost of the whole algorithm as a sum? We will need two nested sums to express the cost.

$$f(n) = 1 + \sum_{i=1}^{n-1} \left[3 + \sum_{j=1}^{n-i} 7 + 3 \right] + 4$$

Simplify the Sum

$$f(n) = 1 + \sum_{i=1}^{n-1}\left[3 + \sum_{j=1}^{n-i} 7 + 3\right] + 4$$

$$= 5 + \sum_{i=1}^{n-1}\left[6 + 7\sum_{j=1}^{n-i} 1\right]$$

$$= 5 + \sum_{i=1}^{n-1}[6 + 7(n-i)]$$

$$= 5 + \sum_{i=1}^{n-1}[6 + 7n - 7i]$$

$$= 5 + \sum_{i=1}^{n-1} 6 + \sum_{i=1}^{n-1} 7n - \sum_{i=1}^{n-1} 7i$$

$$= 5 + 6\sum_{i=1}^{n-1} 1 + 7n\sum_{i=1}^{n-1} 1 - 7\sum_{i=1}^{n-1} i$$

$$= 5 + 6(n-1) + 7n(n-1) - 7\sum_{i=1}^{n-1} i$$

$$= 5 + 6n - 6 + 7n^2 - 7n - \frac{7n(n-1)}{2}$$

$$= 5 + 6n - 6 + 7n^2 - 7n - \frac{7}{2}n^2 + \frac{7}{2}n$$

$$= \frac{7}{2}n^2 + \frac{5}{2}n - 1$$

Tight Bounds

Theorem: $f(n) = \frac{7}{2}n^2 + \frac{5}{2}n - 1 \in \Theta(n^2)$

Proof: We will use the limit test.

$$\lim_{n\to\infty} \frac{\frac{7}{2}n^2 + \frac{5}{2}n - 1}{n^2} = \lim_{n\to\infty} \left(\frac{\frac{7}{2}n^2}{n^2} + \frac{\frac{5}{2}n}{n^2} - \frac{1}{n^2} \right)$$

$$= \lim_{n\to\infty} \left(\frac{7}{2}\right) + \lim_{n\to\infty} \left(\frac{5}{2n}\right) - \lim_{n\to\infty} \left(\frac{1}{n^2}\right)$$

$$= \frac{7}{2} + 0 - 0$$

$$= \frac{7}{2}$$

So by the limit test $f(n) = \frac{7}{2}n^2 + \frac{5}{2}n - 1 \in \Theta(n^2)$. ■

TCSS 321 – Discrete Structures

Selection Sort – Proof of Correctness

Selection Sort

Sort a List
Input: A sequence of integers $A = (a_1, a_2, \ldots, a_n)$.
Output: A' a permutation of A such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

- Another simple sorting algorithm is called selection sort.

```
SelectionSort (A = (a_1, a_2, ..., a_n))
        For i = 1 to n − 1:
                Let m = i.
                For j = i + 1 to n:
                        If a_j < a_m then let m = j.
                Endfor
                Swap(i, m).
        Endfor
Return.
```

Selection Sort

```
SelectionSort (A = (a_1, a_2, ..., a_n))
    For i = 1 to n − 1:
        Let m = i.
        For j = i + 1 to n:
            If a_j < a_m then let m = j.
        Endfor
        Swap(i, m).
    Endfor
Return.
```

- There are two loops in this algorithm. Can we identify the two loop invariants?

Inner Loop

Theorem: After iteration j of the inner loop m is the index of the minimum element of $(a_i, a_{i+1}, \dots, a_j)$.

Proof: (by induction on j)

Base case: ($j = i$)

The inner loop begins at $j = i + 1$ so $j = i$ is prior to the loop. Prior to the loop $m = i$ which is the index of the minimum element of (a_i).

Inductive Hypothesis: Assume prior to iteration $j + 1$ that m is the index of the minimum element of $(a_i, a_{i+1}, \dots, a_j)$.

Inductive case: ($j \geq i$)

Prior to iteration $j + 1$ it is the case that m is the index of the minimum element of $(a_i, a_{i+1}, \dots, a_j)$.

Case 1: $a_{j+1} \geq a_m$

In this case we make no change to m so we can see that m is the index of the minimum element of $(a_i, a_{i+1}, \dots, a_{j+1})$.

Case 2: $a_{j+1} < a_m$

We know that $a_{j+1} < a_m \leq (a_i, a_{i+1}, \dots, a_j)$. In this case we set $m = j + 1$. So we can see that m is the index of the minimum element of $(a_i, a_{i+1}, \dots, a_{j+1})$.

So by induction we have show that after iteration j of the inner loop m is the index of the minimum element of $(a_i, a_{i+1}, \dots, a_j)$. ■

Outer Loop

Theorem: After iteration i of the outer loop the i minimum elements of $(a_1, a_2, \dots, a_n)$ are in ascending order in positions 1 to i.

Proof: (by induction on i)

Base case: ($i = 0$)

Our loop begins at $i = 1$ so $i = 0$ is prior to the loop. In this case it is trivial to see that the 0 minimum elements of $(a_1, a_2, \dots, a_n)$ are in ascending order in positions 1 to 0.

Inductive Hypothesis: Assume prior to iteration $i + 1$ that the i minimum elements of $(a_1, a_2, \dots, a_n)$ are in ascending order in positions 1 to i.

Inductive step: ($i \geq 0$)

We know prior to iteration $i + 1$ that the i minimum elements of $(a_1, a_2, \dots, a_n)$ are in ascending order in positions 1 to i. We need to show that after this iteration the $i + 1^{st}$ minimum element is moved to position $i + 1$.

By the inner loop invariant after iteration j of the inner loop m is the index of the minimum element of $(a_{i+1}, a_{i+2} \dots, a_j)$. In particular, after iteration n of the inner loop m is the index of the minimum element of $(a_{i+1}, a_{i+2}, \dots, a_n)$. Since the i minimum elements of $(a_1, a_2, \dots, a_n)$ are in ascending order in positions 1 to i we know that m is the index of the $i + 1^{st}$ minimum element. When the inner loop completes, we swap the elements in position m and $i + 1$ moving the $i + 1^{st}$ minimum element into position $i + 1$.

So by induction after iteration i of the outer loop the i minimum elements of $(a_1, a_2, \dots, a_n)$ are in ascending order in positions 1 to i.

■

Proof of Correctness

Theorem: The Selection Sort algorithm is correct for the Sort problem.

Proof: Assume an arbitrary input sequence $A = (a_1, a_2, \ldots, a_n)$.

By the outer loop invariant we know that after iteration i of the outer loop the i minimum elements of $(a_1, a_2, \ldots, a_n)$ are in ascending order in positions 1 to i.

In particular, after iteration $n-1$ of the outer loop the $n-1$ maximum elements of $(a_1, a_2, \ldots, a_n)$ are in ascending order in positions 1 to $n-1$. The remaining element is the maximum element, and it is in position n.

We return after iteration $n-1$ so we return a permutation of A in ascending order. ■

TCSS 321 – Discrete Structures

Selection Sort – Run Time Analysis

Selection Sort

```
SelectionSort (A = (a_1, a_2, ..., a_n))
        For i = 1 to n − 1:
                Let m = i.
                For j = i + 1 to n:
                        If a_j < a_m then let m = j.
                Endfor
                Swap(i, m).
        Endfor
Return.
```

1 method call
1 ass/inc, 1 sub and 1 comp
1 ass
1 ass/inc, (1 add) and 1 comp
1 comp and 1 ass
2 operations
1 swap
3 operations
1 return

- How many operations are carried out on each line of this algorithm?
- Can we express the cost of the whole algorithm as a sum? We will need two nested sums to express the cost.

$$f(n) = 1 + \sum_{i=1}^{n-1} \left[5 + \sum_{j=i+1}^{n} 4 + 3 \right] + 4$$

Simplify the Sum

$$f(n) = 1 + \sum_{i=1}^{n-1}\left[5 + \sum_{j=i+1}^{n} 4 + 3\right] + 4$$

$$= 5 + \sum_{i=1}^{n-1}\left[8 + 4\sum_{j=i+1}^{n} 1\right]$$

$$= 5 + \sum_{i=1}^{n-1}[8 + 4(n - (i+1) + 1)]$$

$$= 5 + \sum_{i=1}^{n-1}[8 + 4(n - i)]$$

$$= 5 + \sum_{i=1}^{n-1}[8 + 4n - 4i]$$

$$= 5 + \sum_{i=1}^{n-1} 8 + \sum_{i=1}^{n-1} 4n - \sum_{i=1}^{n-1} 4i$$

$$= 5 + 8\sum_{i=1}^{n-1} 1 + 4n\sum_{i=1}^{n-1} 1 - 4\sum_{i=1}^{n-1} i$$

$$= 5 + 8(n-1) + 4n(n-1) - 4\frac{n(n-1)}{2}$$

$$= 5 + 8n - 8 + 4n^2 - 4n - 2n^2 + 2n$$

$$= 2n^2 + 6n - 3$$

Tight Bounds

Theorem: $f(n) = 2n^2 + 6n - 3 \in \Theta(n^2)$

Proof: We will use the limit test.

$$\lim_{n\to\infty} \frac{2n^2 + 6n - 3}{n^2} = \lim_{n\to\infty} \left(\frac{2n^2}{n^2} + \frac{6n}{n^2} - \frac{3}{n^2} \right)$$

$$= \lim_{n\to\infty} (2) + \lim_{n\to\infty} \left(\frac{6}{n} \right) - \lim_{n\to\infty} \left(\frac{3}{n^2} \right)$$

$$= 2 + 0 - 0$$

$$= 2$$

So by the limit test $f(n) = 2n^2 + 6n - 3 \in \Theta(n^2)$. ■